The Vercel AI SDK: A Framework-Agnostic Deep Dive (AI SDK 6, June 2026)

TL;DR

- The Vercel AI SDK is an Apache-2.0-licensed TypeScript toolkit (npm package `ai`); the current major is **AI SDK 6** (`ai@6.0.x` — 6.0.199 per the vercel/ai GitHub releases page dated 9 Jun, 6.0.202 listed on npm). Its core layer (AI SDK Core) is fully framework-agnostic and runs in any JS runtime — Node.js 18+, Deno, Bun, edge — with zero Vercel hosting lock-in.
- It gives you one unified API (`generateText`, `streamText`, `generateObject`/`streamObject`, `embed`/`embedMany`, tools, the `ToolLoopAgent` agent loop, MCP client, middleware, image/speech/transcription) across providers; you can point it at sovereign self-hosted inference (Ollama, vLLM, llama.cpp, LM Studio) via `@ai-sdk/openai-compatible` using direct provider keys, no gateway required.
- For a clean-room backend: `npm i ai @ai-sdk/anthropic @ai-sdk/openai @ai-sdk/openai-compatible zod`, set `ANTHROPIC_API_KEY`/`OPENAI_API_KEY`, and you have working `generateText`/streaming in a plain Node script or any Hono/Express/Fastify server in minutes.

Key Findings

- **Version reality check:** Despite stale knowledge suggesting v4/v5, Vercel's official blog announced **AI SDK 5 on July 31 2025** ("v5 is a stable, production release"), and **v6 followed in late 2025** with further additions; as of June 2026 both `ai@6.0.x` and a maintained `ai@5.0.x` line are published in parallel (GitHub releases shows `ai@6.0.199` and `ai@5.0.197` published the same time). Use v6 for new projects.
- **The Agent class was renamed.** v5's `Experimental_Agent` is now the stable **`ToolLoopAgent`** in v6, and `system` → `instructions`. `Agent` is now an *interface* you can implement (e.g. Workflow DevKit's `DurableAgent`). AI SDK + 2
- **Tool API changed across v5/v6:** use `inputSchema` (not `parameters`), and control multi-step loops with `stopWhen: stepCountIs(n)` (not `maxSteps`).
- **Sovereignty is well-supported:** direct provider packages auto-read env keys, the AI Gateway is entirely optional, and `@ai-sdk/openai-compatible` cleanly targets local inference servers. License is Apache-2.0.
- **v6 deprecations to note:** `generateObject`/`streamObject` are now *deprecated* in favor of `generateText`/`streamText` with an `output` setting (though still fully functional); `CoreMessage` removed in favor of `ModelMessage`; `convertToCoreMessages` → `convertToModelMessages` (now async). AI SDK

Details

1. Architecture & Philosophy

The AI SDK is "the AI Toolkit for TypeScript… a free open-source library for building AI-powered applications and agents" (vercel/ai). It is organized in layers:

- **AI SDK Core** (`ai` package): the unified, framework-agnostic API for text/object generation, embeddings, tools, agents, image/speech. This is what a backend/agent engineer uses. It runs in *any* JavaScript runtime — Node.js, Deno, Bun, edge runtimes, plain backend services — because it depends only on standard web primitives (`fetch`, `ReadableStream`, SSE).
- **AI SDK UI** (`@ai-sdk/react`, `@ai-sdk/vue`, `@ai-sdk/svelte`, `@ai-sdk/angular`): framework hooks (`useChat`, `useCompletion`, `useObject`). *Optional* — irrelevant for headless backends.
- **Provider packages** (`@ai-sdk/*`): each adapts a vendor API to the SDK's Language Model Specification (currently the v3 spec in SDK 6; the underlying model interface evolved to LanguageModelV2 in v5).

The unified provider abstraction model. Every provider package exposes a factory (`openai('gpt-5.4')`, `anthropic('claude-opus-4-6')`) returning a `LanguageModel` that conforms to a single spec. Switching providers is a one-line change. The v5 LanguageModelV2 redesign made all model outputs "content parts" (text, reasoning, tool calls, sources, files) in one ordered array, which is why reasoning models, multimodal, and computer-use agents all work through one interface. `npm`

Package structure / first-party providers (all under `@ai-sdk/`): `openai`, `anthropic`, `google` (Generative AI), `google-vertex`, `mistral`, `groq`, `amazon-bedrock`, `azure`, `xai` (Grok), `deepseek`, `togetherai`, `cohere`, `fireworks`, `cerebras`, `deepinfra`, `perplexity`, `replicate`, `fal`, `luma`, `elevenlabs`, `assemblyai`, `deepgram`, `gladia`, `lmnt`, `hume`, `revai`, `baseten`, `huggingface`, `vercel` (v0), plus the crucial **`@ai-sdk/openai-compatible`** generic adapter and **`@ai-sdk/gateway`**.

Community / self-hostable providers (implement the Language Model Specification): **Ollama** (`ollama-ai-provider-v2` and `ai-sdk-ollama` — the latter is a v6 provider built on the official `ollama` package, with `ai-sdk-ollama@^2` for v5), **llama.cpp**, **LM Studio** and **NVIDIA NIM** (documented under the openai-compatible umbrella), Cloudflare Workers AI, OpenRouter, Portkey, FriendliAI, LangDB, Browser AI (WebLLM/Transformers.js for in-browser models), and many more. For the sovereignty-minded: *any* server implementing the OpenAI API spec works through `@ai-sdk/openai-compatible` with no dedicated package.

v4 → v5 → v6 breaking changes (the ones that matter for backend code):

- **v4 → v5** (July 31 2025, major architectural overhaul): `UIMessage` and `ModelMessage` became separate types (conversion is now explicit via `convertToModelMessages`); streaming switched from a custom protocol to native **Server-Sent Events**; tools use `inputSchema`/`outputSchema` instead of `parameters`/`result`; `maxTokens` → `maxOutputTokens`; multi-step uses `stopWhen` (the `maxSteps` parameter was removed from `useChat`); `.reasoning` → `.reasoningText`; `mimeType` → `mediaType`; new `Experimental_Agent` class; speech/transcription added; Zod 4 supported. Codemods: `npx @ai-sdk/codemod@latest migrate`. (Vercel)
- **v5 → v6** (late 2025): `Experimental_Agent` → **`ToolLoopAgent`** (and `system` → `instructions`); **`generateObject`**/**`streamObject`** **deprecated** in favor of `generateText`/`streamText` + `output`; **`CoreMessage`** **removed** (use `ModelMessage`); `convertToCoreMessages` → `convertToModelMessages` (now **async**); embedding methods `textEmbeddingModel`/`textEmbedding` → `embeddingModel`/`embedding`; `strictJsonSchema` on by default; `structuredOutputs` provider option removed (use `strictJsonSchema`); Azure provider defaults to the Responses API; tool UI helper renames (`isToolUIPart` → `isStaticToolUIPart`, etc.); a deprecation-warning logger (disable with `AI_SDK_LOG_WARNINGS=false`). Vercel describes v6 migration as "intentionally simple" with codemods (`npx @ai-sdk/codemod upgrade`). (AI SDK + 3)

Node requirement: AI SDK 6 requires **Node.js 18+**. The official Getting Started: Node.js docs state "Node.js 18+ and pnpm installed on your local development machine," the npm README states "You will need Node.js 18+," and the repo `package.json` engines field is `"node": "^18.0.0 || ^20.0.0 || ^22.0.0"` — so Node 18/20/22 are the supported majors with 18 as the floor. (AI SDK) (npm)

2. Detailed Capability List

Text generation & streaming. `generateText({ model, prompt | messages, system, tools, stopWhen, ... })` returns `{ text, reasoning, reasoningText, steps, toolCalls, toolResults, usage, finishReason, ... }`. `streamText(...)` returns a result exposing `textStream` (async iterable of text chunks), `fullStream` (typed parts: text-delta, reasoning, tool-input-start/delta, tool-call, tool-result, source, finish), plus `onChunk`, `onFinish`, `onError`, `onStepFinish`, and experimental lifecycle callbacks (`experimental_onStart`, `experimental_onStepStart`, `experimental_onToolCallStart`/`Finish`). Streaming uses backpressure — you must consume the stream for it to finish. Response helpers: `toUIMessageStreamResponse()`, `pipeUIMessageStreamToResponse(res)`, `toTextStreamResponse()`, `pipeTextStreamToResponse(res)`. (AI SDK + 2)

Structured output. Two routes: (a) the still-functional but v6-deprecated `generateObject`/`streamObject({ model, schema, output })` with `output: 'object' | 'array' | 'enum' | 'no-schema'`; (b) the v6-preferred `generateText`/`streamText` with the

`output` setting and the `Output` helper: `Output.object({ schema })`, `Output.array({ element })`, `Output.choice({ options })` (enum/classification), `Output.json()` (unstructured). Schemas may be Zod, Valibot, or JSON Schema (`jsonSchema()`). `streamObject`/array exposes `partialObjectStream` and `elementStream` (each element validated as it completes). Crucial caveat: structured output **counts as a step**, so when combining with tools, increase `stopWhen` accordingly.

Tool calling / function calling. Define tools with the `tool()` helper (needed for TypeScript to infer `execute` arg types from `inputSchema`):

```ts
weather: tool({
  description: 'Get the weather in a location',
  inputSchema: z.object({ location: z.string() }),
  execute: async ({ location }, { abortSignal }) => ({ location, temperature: 72 }),
})
```

`execute` is optional (omit to forward calls to a client/queue). v6 adds: `needsApproval` (human-in-the-loop, boolean or function of input), per-tool `strict` mode, `toModelOutput` for flexible tool outputs, input examples, and `dynamicTool()` for runtime-defined tools. Multi-step loops: `stopWhen` accepts `stepCountIs(n)` (default `stepCountIs(20)`), `hasToolCall(name)`, `isLoopFinished()` (no limit), custom `StopCondition` functions, or an array (stops on any). `toolChoice: 'auto' | 'required' | 'none' | { type:'tool', toolName }`. Provider-executed tools exist (web search, code execution, memory, computer use). The abort signal is forwarded into `execute`. AI SDK

Agents. `ToolLoopAgent` (v6) encapsulates model + instructions + tools + loop control into a reusable object usable across chat UIs, background jobs, API endpoints, and CLI daemons: dplooy

```ts
const agent = new ToolLoopAgent({ model, instructions, tools, stopWhen: stepCountIs(1
const result = await agent.generate({ prompt });       // GenerateTextResult
const stream = agent.stream({ prompt });                // StreamTextResult
```

Loop control: `stopWhen` (when to stop) and **`prepareStep`** (called before each step; can change model, tools, toolChoice, messages — used for context compression, model-switching by complexity, dynamic tool gating). v6 also adds `callOptionsSchema` + `prepareCall` for type-safe per-call options (e.g. inject RAG context once per call, select model by tier). Subagents are just a `ToolLoopAgent` invoked inside another agent's tool `execute`. For full control, hand-roll the loop with `generateText` + your own while-loop. AI SDK Upstash

Embeddings & RAG. `embed({ model, value })` → `{ embedding, usage }`; `embedMany({ model, values, maxParallelCalls })` → `{ embeddings, usage }` (auto-chunks large batches). `cosineSimilarity(a, b)` for ranking. v6 also adds a `rerank()` function. The canonical mini-RAG pattern: chunk → `embedMany` → store `{embedding, value}` → at query time `embed` the query, sort chunks by `cosineSimilarity`, inject top-k into the prompt. Production guides use pgvector or Upstash Vector as the store.

Image generation. `generateImage({ model, prompt, size })` → `{ images }` (experimental, also exported as `experimental_generateImage`). v6 adds image editing/inpainting (`prompt: { text, images, mask }`) via the openai-compatible provider's `/images/edits`. ai-sdk

Speech & transcription (experimental). `generateSpeech({ model: openai.speech('tts-1'), text, voice })` → `{ audio }`; `transcribe({ model: openai.transcription('whisper-1'), audio })` → `{ text, segments, language, durationInSeconds }`. Imported as `experimental_generateSpeech`/`experimental_transcribe`. Providers include OpenAI, ElevenLabs, Deepgram, AssemblyAI, Gladia, LMNT, Hume, Rev.ai. `audio` accepts Uint8Array/ArrayBuffer/Buffer/base64/URL. AI SDK AI SDK

Multimodal inputs. Messages support `ImagePart`, `FilePart` (PDFs, files) alongside `TextPart`; images/files accept `string | Uint8Array | Buffer | ArrayBuffer | URL` with a `mediaType`.

Reasoning models. Configure via `providerOptions`. For Anthropic extended thinking: `providerOptions: { anthropic: { thinking: { type: 'enabled', budgetTokens: 12000 } } }` (an `effort: 'low'|'medium'|'high'` option also exists; both can be combined). Access via destructured `reasoning`/`reasoningText`, or in streaming via `fullStream` parts. For OpenAI o-series/GPT-5: `providerOptions: { openai: { reasoningEffort: 'low', reasoningSummary: 'auto' } }`, with reasoning token counts at `providerMetadata.openai.reasoningTokens`. For models that wrap reasoning in `<think>` tags (DeepSeek R1, Magistral), use `extractReasoningMiddleware({ tagName: 'think' })`: ai-sdk ai-sdk

```ts
const model = wrapLanguageModel({ model: yourModel, middleware: extractReasoningMiddl
const { text, reasoningText } = await generateText({ model, prompt: 'What is 15 * 24?
```

Middleware. `wrapLanguageModel({ model, middleware })` returns an enhanced model. A middleware (type `LanguageModelV3Middleware` in v6) implements any of `transformParams`, `wrapGenerate`, `wrapStream` — model-agnostic logging, caching, guardrails, RAG injection, rate-limiting. Multiple middlewares compose in order (applied innermost-last). Built-ins: `extractReasoningMiddleware`, `simulateStreamingMiddleware`, `defaultSettingsMiddleware`, `addToolInputExamplesMiddleware`, `extractJsonMiddleware`. Community: `@ai-sdk-`

`tool/parser` (`hermesToolMiddleware`, `gemmaToolMiddleware`) adds tool-calling to local models lacking native function calling — directly relevant to self-hosted deployments. AI SDK + 5

Provider registry & custom providers. `createProviderRegistry({ anthropic, openai, … })` lets you reference models by `providerId:modelId` string at runtime (custom separator supported) — useful for runtime model selection, A/B testing, and fallback routing. `customProvider({ languageModels, fallbackProvider })` creates aliases/preconfigured settings and can restrict the model set. AI SDK AI SDK

Telemetry. OpenTelemetry-based via `experimental_telemetry: { isEnabled: true, functionId, recordInputs, recordOutputs }` on any generate/stream call. Emits standard `gen_ai.*` and AI-SDK-specific `ai.*` spans (model calls, `ai.toolCall`, etc.). Works with any OTel backend; for non-Next.js (Express/Fastify/Hono/plain Node) initialize the OTel Node SDK directly (e.g. `@opentelemetry/sdk-node` + an OTLP exporter, or `@pydantic/logfire-node`, or `@langfuse/otel`'s `LangfuseSpanProcessor`). AI SDK npm

Error handling, retries, abort, timeouts. `maxRetries` on all functions; `abortSignal` accepted by generate/stream/embed/transcribe and forwarded into tools; `streamText` puts errors into the stream (use `onError`) rather than throwing, to avoid crashing servers; typed errors (`AI_NoSpeechGeneratedError`, `MCPClientError`, etc.). AI SDK

Streaming protocols & frameless consumption. Native SSE. The **UI Message Stream** protocol (set header `x-vercel-ai-ui-message-stream: v1` for custom backends) carries typed parts; the **text stream** is plain text. To consume *without any frontend framework*: iterate `result.textStream`/`result.fullStream` in a CLI/daemon; or serve over HTTP with `pipeUIMessageStreamToResponse(res)` (Node `http`), `result.toUIMessageStreamResponse()`/`toTextStreamResponse()` (Hono/edge/Web `Response`), or Hono's `stream`/`streamSSE` helpers. `createUIMessageStream({ execute })` + `writer.write`/`writer.merge` lets you emit custom data parts. `readUIMessageStream` converts a chunk stream to an async-iterable of `UIMessage`s on the client side. AI SDK + 3

Prompt management. `system` prompt, `prompt` (string), or `messages` array of `ModelMessage` (`SystemModelMessage`/`UserModelMessage`/`AssistantModelMessage`/`ToolModelMessage`, each with typed content parts). `UIMessage` (client-facing, has a `parts` array) is distinct from `ModelMessage` (sent to the LLM); convert with the async `convertToModelMessages(uiMessages)`.

Caching / rate limiting. Implemented as middleware (cache by hashed params, short-circuit identical calls) or via provider-native prompt caching (Anthropic, Bedrock). The cookbook ships local-caching and dynamic-prompt-caching middleware examples.

bash

```
mkdir my-agent && cd my-agent
npm init -y
npm pkg set type=module
npm i ai @ai-sdk/anthropic @ai-sdk/openai @ai-sdk/openai-compatible zod
npm i -D typescript tsx @types/node
npx tsc --init
```

`tsconfig.json`: use ESM-friendly settings — `"module": "ES2022"`, `"moduleResolution": "Bundler"` (or `"NodeNext"`), `"target": "ES2022"`, `"strict": true`. Node 18+ (20 or 22 recommended). Set env vars — provider packages auto-detect them: `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, etc. (Gateway string-model usage instead reads `AI_GATEWAY_API_KEY`.)

Run scripts with `npx tsx index.ts`. **Deno:** `deno run --allow-net --allow-env npm:tsx index.ts` or import via `npm:ai`. **Bun:** `bun add ai @ai-sdk/anthropic zod` then `bun index.ts`.

4. Code Examples (current v6 API)

Hello world — generateText (plain Node script):

ts

```ts
import { generateText } from 'ai';
import { anthropic } from '@ai-sdk/anthropic';

const { text } = await generateText({
  model: anthropic('claude-opus-4-6'),
  prompt: 'Explain quantum entanglement in two sentences.',
});
console.log(text);
```

streamText — consume in a CLI/daemon:

ts

```ts
import { streamText } from 'ai';
import { openai } from '@ai-sdk/openai';

const result = streamText({
  model: openai('gpt-5.4'),
  prompt: 'Write a haiku about distributed systems.',
});
for await (const chunk of result.textStream) process.stdout.write(chunk);
console.log('\nusage:', await result.usage);
```

Structured output (v6-preferred output setting):

ts

```ts
import { generateText, Output } from 'ai';
import { anthropic } from '@ai-sdk/anthropic';
import { z } from 'zod';

const { output } = await generateText({
  model: anthropic('claude-sonnet-4-5'),
  output: Output.object({
    schema: z.object({
      title: z.string(),
      severity: z.enum(['low', 'medium', 'high']),
      tags: z.array(z.string()),
    }),
  }),
  prompt: 'Classify this incident: database connection pool exhausted under load.',
});
console.log(output);
```

Tool calling + multi-step loop:

ts

```ts
import { generateText, tool, stepCountIs } from 'ai';
import { anthropic } from '@ai-sdk/anthropic';
import { z } from 'zod';

const { text, steps } = await generateText({
  model: anthropic('claude-sonnet-4-5'),
  stopWhen: stepCountIs(5),
  tools: {
    getBalance: tool({
      description: 'Get the on-chain token balance for an address',
      inputSchema: z.object({ address: z.string(), token: z.string() }),
      execute: async ({ address, token }) => ({ address, token, balance: '1234.56' }),
    }),
  },
  prompt: 'What is the USDC balance of 0xabc...?',
});
console.log(text, steps.length);
```

ToolLoopAgent (reusable agent):

ts

```ts
import { ToolLoopAgent, stepCountIs } from 'ai';
import { anthropic } from '@ai-sdk/anthropic';

export const agent = new ToolLoopAgent({
  model: anthropic('claude-sonnet-4-6'),
  instructions: 'You are an autonomous backend agent. Use tools, then answer.',
  tools: { /* ...tools... */ },
  stopWhen: stepCountIs(20),
  prepareStep: ({ stepNumber }) => (stepNumber === 0 ? { toolChoice: 'required' } : {
});

const result = await agent.generate({ prompt: 'Analyze the dataset and summarize.' });
console.log(result.text);
```

Embeddings + cosine similarity mini-RAG:

ts

```ts
import { embed, embedMany, cosineSimilarity, generateText } from 'ai';
import { openai } from '@ai-sdk/openai';

const chunks = essay.split('.').map(s => s.trim()).filter(Boolean);
const { embeddings } = await embedMany({
  model: openai.embeddingModel('text-embedding-3-small'),
  values: chunks,
});
const db = embeddings.map((embedding, i) => ({ embedding, value: chunks[i] }));

const { embedding: q } = await embed({
  model: openai.embeddingModel('text-embedding-3-small'),
  value: 'What did the author say about sovereignty?',
});
const top = db
  .map(d => ({ ...d, score: cosineSimilarity(q, d.embedding) }))
  .sort((a, b) => b.score - a.score)
  .slice(0, 3)
  .map(d => d.value)
  .join('\n');

const { text } = await generateText({
  model: openai('gpt-5.4'),
  system: `Answer using only this context:\n${top}`,
  prompt: 'What did the author say about sovereignty?',
});
console.log(text);
```

Self-hosted inference via openai-compatible (Ollama / vLLM / llama.cpp):

```ts
```

```ts
import { createOpenAICompatible } from '@ai-sdk/openai-compatible';
import { generateText } from 'ai';

const local = createOpenAICompatible({
  name: 'local',
  baseURL: 'http://localhost:11434/v1', // Ollama; vLLM/llama.cpp server: their /v1 U
  apiKey: 'ollama',                     // placeholder; many local servers ignore it
});

const { text } = await generateText({
  model: local('qwen2.5-coder:7b'),
  prompt: 'Refactor this function for readability.',
});
console.log(text);
```

(Note the `/v1` suffix is required — local servers expose the OpenAI-compatible API there, not at their native `/api` path. The dedicated `ai-sdk-ollama` provider is an alternative that adds tool-call reliability and JSON repair on top of the official `ollama` client.)

Provider registry with Anthropic + a local endpoint:

ts

```ts
import { createProviderRegistry, generateText } from 'ai';
import { anthropic } from '@ai-sdk/anthropic';
import { createOpenAICompatible } from '@ai-sdk/openai-compatible';

const registry = createProviderRegistry({
  anthropic,
  local: createOpenAICompatible({ name: 'local', baseURL: 'http://localhost:11434/v1'
});

const { text } = await generateText({
  model: registry.languageModel('local:llama3.3'),   // or 'anthropic:claude-sonnet-4
  prompt: 'Summarize the latest block.',
});
```

Middleware (reasoning extraction for a local DeepSeek-R1):

ts

```ts
import { wrapLanguageModel, extractReasoningMiddleware, generateText } from 'ai';
import { createOpenAICompatible } from '@ai-sdk/openai-compatible';

const local = createOpenAICompatible({ name: 'local', baseURL: 'http://localhost:8000
const model = wrapLanguageModel({
  model: local('deepseek-r1'),
  middleware: extractReasoningMiddleware({ tagName: 'think' }),
});
const { text, reasoningText } = await generateText({ model, prompt: 'What is 15 * 24?
console.log({ reasoningText, text });
```

HTTP endpoint — Hono (Web Response):

ts

```ts
import { serve } from '@hono/node-server';
import { streamText } from 'ai';
import { Hono } from 'hono';

const app = new Hono();
app.post('/', async c => {
  const result = streamText({ model: 'openai/gpt-4o', prompt: 'Invent a holiday.' });
  return result.toUIMessageStreamResponse();
});
serve({ fetch: app.fetch, port: 8080 });
```

HTTP endpoint — plain Node `http` (no framework):

ts

```ts
import { streamText } from 'ai';
import { createServer } from 'http';

createServer(async (req, res) => {
  const result = streamText({ model: 'openai/gpt-4o', prompt: 'Invent a holiday.' });
  result.pipeUIMessageStreamToResponse(res);
}).listen(8080);
```

(Express is identical, swapping in `pipeUIMessageStreamToResponse(res)` inside an `app.post` handler; Fastify and Nest.js have cookbook equivalents.)

MCP client tool usage (stable `@ai-sdk/mcp`):

ts

```ts
import { createMCPClient } from '@ai-sdk/mcp';
import { Experimental_StdioMCPTransport } from '@ai-sdk/mcp/mcp-stdio';
import { generateText, stepCountIs } from 'ai';

const client = await createMCPClient({
  transport: new Experimental_StdioMCPTransport({ command: 'node', args: ['server.js'
  // or HTTP: transport: { type: 'http', url: 'http://localhost:3000/mcp', headers: {
});
try {
  const tools = await client.tools();
  const { text } = await generateText({
    model: 'anthropic/claude-sonnet-4.5',
    tools,
    stopWhen: stepCountIs(10),
    prompt: 'Use the available tools to answer.',
  });
  console.log(text);
} finally {
  await client.close();
}
```

MCP transports: stdio (local), HTTP (Streamable HTTP), SSE; OAuth supported on HTTP/SSE via `authProvider`. The MCP client (`@ai-sdk/mcp`, ~v1.0.x) is lightweight (tool conversion, resources, prompts, elicitation) but does not yet do session management/resumable streams. MintMCP + 2

5. Ecosystem & Context

- **License:** Apache-2.0 (confirmed on the npm package) — aligns with the user's standards. Fully open source: the ai-sdk.dev homepage states **"12.5M Weekly downloads · 24.8K GitHub stars · 658+ Contributors"** (with ~4.6k forks per the GitHub releases page); ai-sdk.guide reports "over 30 million combined weekly npm installs across the ai core package and @ai-sdk/* providers." Vercel cites 20M+ monthly downloads.
- **No Vercel lock-in.** The SDK requires no Vercel hosting. Two model-addressing modes: pass a *string* like `'anthropic/claude-opus-4.6'` (routes through the Vercel AI Gateway, needing `AI_GATEWAY_API_KEY`), or pass a *provider instance* like `anthropic('claude-opus-4-6')` that talks **directly** to the vendor with your own key. The Gateway is purely optional convenience: per Vercel's own docs, "AI Gateway reflects provider pricing with no markup and does not charge a platform fee on inference, including on Bring Your Own Key (BYOK)

requests." For sovereign deployments, use provider instances or `@ai-sdk/openai-compatible` against self-hosted servers and the Gateway never enters the picture. `npm`

- **vs LangChain.js:** the AI SDK is a focused, strongly-typed TypeScript abstraction over providers + streaming + tools + a lightweight agent loop; LangChain.js offers broader, heavier orchestration abstractions. Downstream agent frameworks (Mastra, Inngest agent kit, even LangChain's TS port) increasingly integrate *against* the AI SDK rather than competing. vs direct provider SDKs: you trade a thin abstraction for provider portability, standardized streaming, and typed tools — generally worth it for multi-provider/at-scale backends.
- **MCP status:** First-class via `@ai-sdk/mcp` (`createMCPClient`); v6 emphasizes "full MCP support." MCP tools become AI SDK tools transparently.
- **Crypto / autonomous-agent angle:** The `ToolLoopAgent` running headless in a backend is the natural home for autonomous on-chain agents. **x402** is an HTTP-402 stablecoin micropayment protocol for machine-to-machine payments; the **x402 Foundation launched April 2 2026** under the Linux Foundation (Coinbase contributed the protocol), with named participants including AWS, Google, Visa, Mastercard, Stripe, Circle, Cloudflare, Coinbase, Shopify, Microsoft, Solana Foundation, and Polygon Labs. (Note: Vercel is *not* in the Linux Foundation's named participant list, though it has separately shipped x402 middleware for its serverless functions.) Linux Foundation CEO Jim Zemlin: "The x402 Foundation will create an open, community-governed home to develop these capabilities in the open, ensuring they evolve with transparency, interoperability, and broad participation across the ecosystem." x402 pairs naturally with AI SDK tools — community packages like `x402-agent-tools` expose paid endpoints "as Vercel-compatible tool objects with automatic x402 payment handling," so your agent calls a `tool()`, and the underlying fetch handles the 402 → sign USDC → retry flow without API keys. Self-hosted models via the openai-compatible provider close the sovereignty loop: an agent can run on local inference and pay for external data per request.

Recommendations

1. **Start now on v6** with `npm i ai @ai-sdk/anthropic @ai-sdk/openai @ai-sdk/openai-compatible zod`. Pin exact versions for any `experimental_*` API (image/speech/transcription, telemetry, MCP transport). Verify your runtime is Node 18+ (prefer 20/22).
2. **For sovereignty:** default to provider instances + your own keys, or `@ai-sdk/openai-compatible` against Ollama/vLLM/llama.cpp. Avoid the string-model/Gateway path unless you explicitly want it. Add `extractReasoningMiddleware` for `<think>`-tag local models and `@ai-sdk-tool/parser` middleware for local models lacking native tool calling.
3. **Structure agents** with `ToolLoopAgent` defined once and reused across CLI/daemon/HTTP. Use `stopWhen` + `prepareStep` for budget/loop safety; set a real step cap (never run

unbounded `isLoopFinished()` in production without a token-budget stop condition).

4. **Observability:** enable `experimental_telemetry` and wire an OTel backend (Langfuse/Logfire/SigNoz). For headless services initialize `@opentelemetry/sdk-node` yourself (the `@vercel/otel` one-liner is Next.js-only). Pydantic
5. **Thresholds that change the plan:** if you need resumable/durable agent runs, adopt the `Agent` interface with Workflow DevKit's `DurableAgent` rather than raw `ToolLoopAgent`; if you outgrow in-memory RAG, move `embedMany` output into pgvector/Upstash; if you need many providers with failover and accept the dependency, the Gateway (zero markup, BYOK) becomes worthwhile.

Caveats

- **Version drift in third-party content is severe.** Many 2026 blogs still show v4/v5 patterns (`parameters`, `maxSteps`, `Experimental_Agent`, `system` on agents). Always cross-check against ai-sdk.dev (v6) and the GitHub changelog. Some sources speculatively reference "v7 in active beta" and far-future model names (e.g. `claude-opus-4.7`, `gpt-5.4`, `gemini-3-flash`); treat unreleased version/model claims as forward-looking, not confirmed. The exact latest patch differs slightly between sources (GitHub releases `6.0.199`; npm `6.0.202`) because npm may be hours fresher. Pydantic
- **Experimental surfaces change without semver guarantees:** image generation, speech, transcription, telemetry, MCP stdio transport, and some agent UI stream helpers are `experimental_`-prefixed. Pin versions.
- **Streaming reasoning part-type naming varies** between docs (`part.type === 'reasoning'` with `part.textDelta` vs `'reasoning-delta'` with `part.text`); verify against your installed version.
- **`generateObject`/`streamObject` are deprecated in v6** (still work) — new code should prefer `generateText`/`streamText` with `Output`.
- Community providers (Ollama, etc.) are "provider dependent" in feature coverage; not all support tools, structured outputs, or multimodal. Verify per model.